CS415/515 Milestone 2 Report

BenchTime: A lab equipment reservation system

Luke McArthur & Jesse Seidel

lgmcarthur@crimson.ua.edu & jpseidel@crimson.ua.edu

Github Link: https://github.com/luke-mca/BenchTime/tree/main

Submitted November 2nd, 2026

Changes Made Since Milestone 1

Many of the changes made since milestone 1 have been focused on testing. However some new features have been added including:

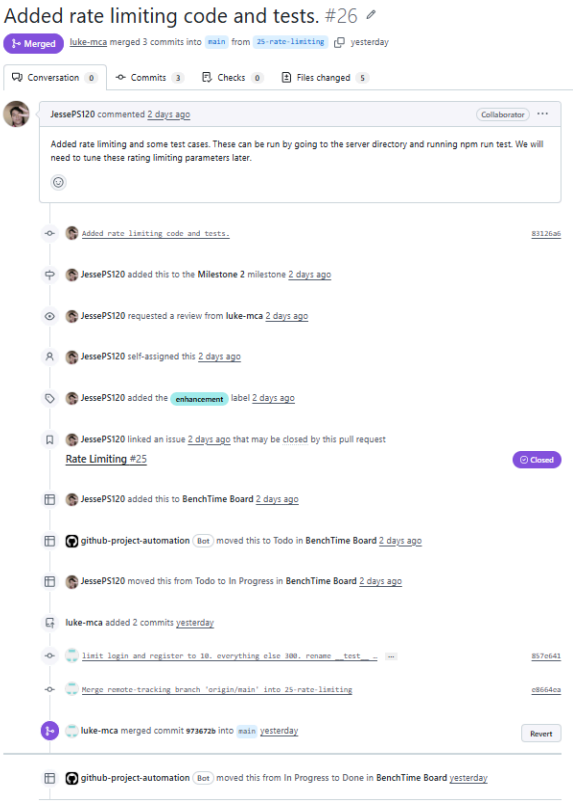
- Utilization statistics for managers allow managers to view utilization statistics for pieces of equipment after a reservation has passed.
- Expanded rights for managers. Managers can now edit/delete reservations belonging to any member of their lab. This allows them to manage their lab easier.

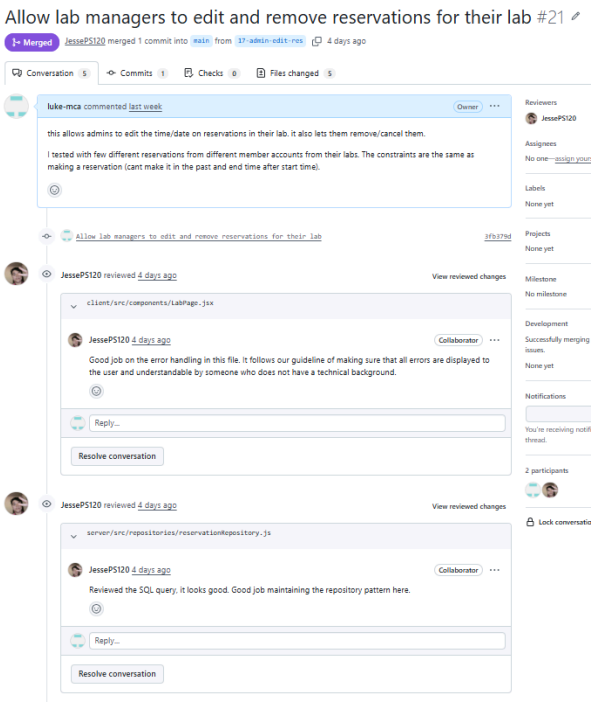
For testing, the following areas have been covered. The rest of this document goes into more detail for each of these areas.

- Unit testing.
- Component testing.
- Integration testing.
- Security/rate limit testing.
- Flow/viewport testing.
- Any necessary document updates(Quality assurance plan/CHANGELOG etc).

Peer Code Review

Every change goes through a pull request from an issue branch, and the other team member reviews it before merging. The reviewer reads the diff, runs the feature locally and checks it against our Definition of Done, including error handling, the repository pattern and test coverage. Feedback is left as PR comments or a GitHub review.





Above are some examples:

PR #21: https://github.com/luke-mca/BenchTime/pull/21
PR #26: https://github.com/luke-mca/BenchTime/pull/26

Some early PRs were merged without written review by the other developer. For Milestone 3, we'll add the CI pipeline that will block merges until checks pass.

Quality Assurance Plan

Our quality assurance plan can be found on the repository here under the docs/Quality folder. This quality assurance plan covers the following areas:

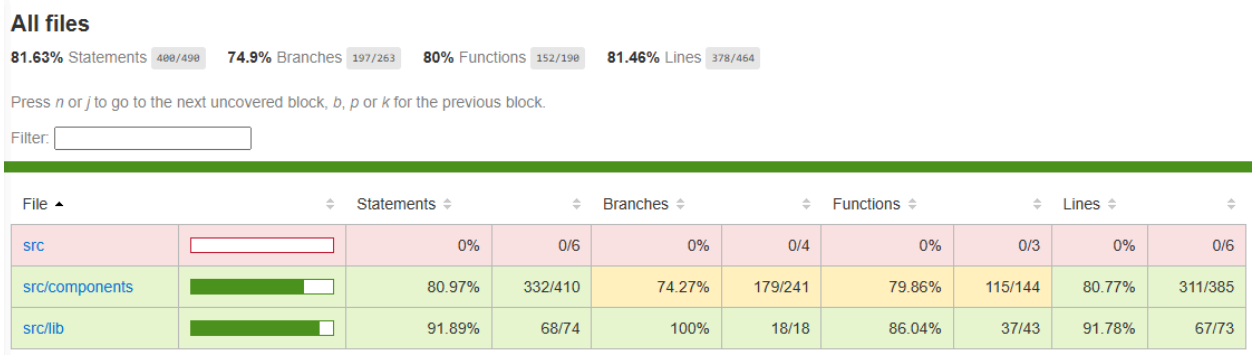
- Standards
- Reviews
- Testing
- Defect Tracking
- Metrics Tracked
- Ownership

Unit Tests and Coverage

BenchTime has 237 unit tests using Vitest. 126 are for the server and 111 for the client. These unit tests run without a database, giving the developer quick feedback. Server tests mock the repository layer to check business rules, permissions and status codes. Client side tests use React Testing Library to interact with components as a user does with the API layer mocked.

To run these tests, use ‘npm test’ in the server and client directories. To generate a coverage report, run ‘npm run test:coverage’.

Client Coverage Report:



All files

81.63% Statements 400/490 **74.9%** Branches 197/263 **80%** Functions 152/190 **81.46%** Lines 378/464

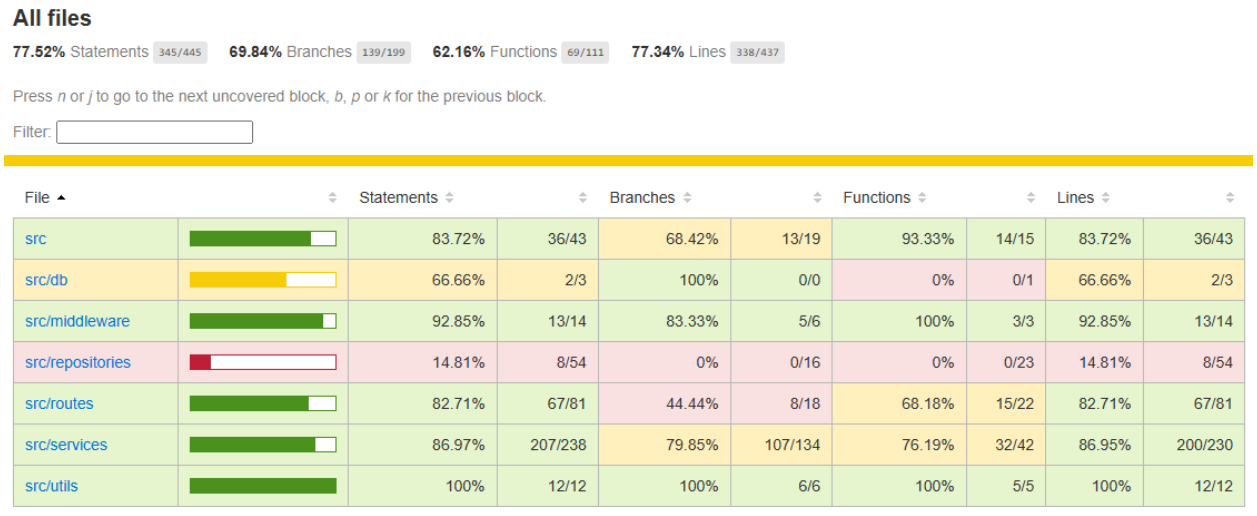
Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

File	Statements		Branches		Functions		Lines	
src	0%	0/6	0%	0/4	0%	0/3	0%	0/6
src/components	80.97%	332/410	74.27%	179/241	79.86%	115/144	80.77%	311/385
src/lib	91.89%	68/74	100%	18/18	86.04%	37/43	91.78%	67/73

’src’ has no tests because it is only used to set up routes and page layout.

Server Coverage Report:



All files

77.52% Statements 345/445 **69.84%** Branches 139/199 **62.16%** Functions 69/111 **77.34%** Lines 338/437

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

File	Statements		Branches		Functions		Lines	
src	83.72%	36/43	68.42%	13/19	93.33%	14/15	83.72%	36/43
src/db	66.66%	2/3	100%	0/0	0%	0/1	66.66%	2/3
src/middleware	92.85%	13/14	83.33%	5/6	100%	3/3	92.85%	13/14
src/repositories	14.81%	8/54	0%	0/16	0%	0/23	14.81%	8/54
src/routes	82.71%	67/81	44.44%	8/18	68.18%	15/22	82.71%	67/81
src/services	86.97%	207/238	79.85%	107/134	76.19%	32/42	86.95%	200/230
src/utils	100%	12/12	100%	6/6	100%	5/5	100%	12/12

‘src/repositories’ is low because these files are mostly SQL that are mocked for the unit tests.

Overage Coverage

For the client and the server we aim to have around 75% - 80% code line coverage for the unit tests. The reason for this is that some of the code cannot be fully covered in simple unit tests, such as code that belongs to the repositories which must be mocked. This is required to keep the unit tests streamlined and simple, and is something that is instead covered in the integration/viewport tests where an actual database is used. The rest of the remaining code is hard to trigger during an automatic test and is otherwise low risk. An example of this can be seen in lines 64-80 of the errorHandler in app.js where an error code 500 is only triggered when an internal server error occurs and thus is hard to emulate. For milestone 3, we will work towards further increasing the number of lines covered to meet the 80% goal for the unit tests then cover everything else in our integration/viewport tests.

Integration Tests

Integration tests are used to test how everything that the unit tests checks works when put together. They use the real endpoints and a real test database (benchtime_test). The test database ensures that tests do not affect a developer's local development database. The tests clear the tables before each test so that they are independent of each other.

Server flow tests (17 total tests):

Flow	Tests	What is checked
Core	2	The manager creates a lab, adds equipment and a member. The member logs in, reserves equipment, then cancels. The manager can see the user's reservation. Logging out.
Access	6	Tests that other managers and non members can't access a lab or equipment they aren't part of. Tests that members can't perform manager actions. Removed members lose access and signed out users are rejected.
Database rules	9	Duplicate usernames, lab names, equipment names and memberships are rejected. Checks that overlapping reservations aren't allowed. Deleting lab deletes its members, equipment, and reservations.

Browser flow test:

This test uses Playwright to run the core flow from the real UI the user uses. It tests this flow at mobile and desktop dimensions. The test runs the same core flow that is used in the server test. This type of integration test makes sure that everything is connected properly in the client as well as the server.

To run the tests:

Server: 'npm run test:integration'

Browser Flow: 'npx playwright test flow.spec.js --project=mobile --project=desktop'

Results: All 17 server tests passed. Browser flow passed at both sizes.

```
✓ __integration__/flows.test.js (17 tests) 9261ms
  ✓ core flow (2)
    ✓ lets a manager set up a lab and a member reserve equipment 1121ms
    ✓ ends the session on log out 488ms
  ✓ access control (6)
    ✓ hides a lab from another manager 710ms
    ✓ hides a lab from a member who was not added 902ms
    ✓ keeps manager actions from members of the lab 494ms
    ✓ does not let a member cancel someone else's reservation 654ms
    ✓ takes away lab access from a removed member but keeps their account 626ms
    ✓ requires the user to be signed in 352ms
  ✓ database rules (9)
    ✓ rejects a username that differs only by case 309ms
    ✓ rejects a duplicate lab name for the same manager only 435ms
    ✓ rejects a duplicate equipment name until the first one is removed 392ms
    ✓ rejects adding the same member twice 506ms
    ✓ reservation times (4)
      ✓ rejects a reservation that overlaps another 408ms
      ✓ allows back-to-back reservations 368ms
      ✓ allows the same time on different equipment 523ms
      ✓ rejects an edit that overlaps another reservation 408ms
    ✓ deletes a lab's members, equipment and reservations with it but keeps the accounts 556ms

Test Files  1 passed (1)
     Tests  17 passed (17)
  Start at  01:41:42
  Duration  11.13s (tests 86%, import 10%, transform 4%)
```

```
Running 2 tests using 2 workers

  ✓  1 …_test__\flow.spec.js:42:1 › a manager sets up a lab and a member reserves equipment (10.4s)
  ✓  2 …_test__\flow.spec.js:42:1 › a manager sets up a lab and a member reserves equipment (10.3s)

> server@1.0.0 db:reset:test
> node --env-file=.env.test scripts/resetTestDb.js

Database is at version 3.
Cleared all tables in benchtime_test.

  2 passed (31.8s)
```

Cross Platform/Viewport Testing

Viewport testing is done using the playwright framework and currently tests the following browsers:

- Chromium based
- Firefox
- Webkit/Apple Safari

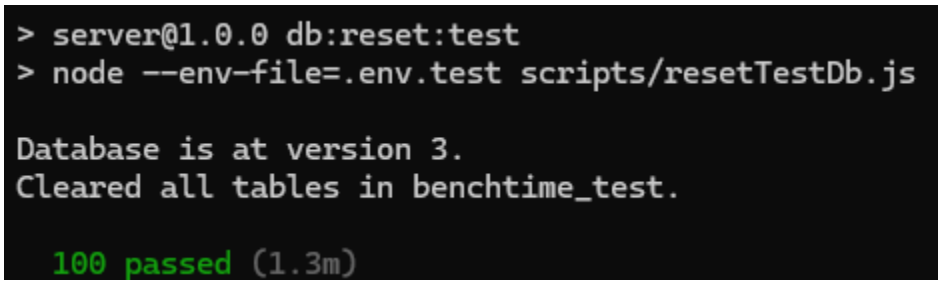
The following devices/viewports are tested:

- Desktop Chrome
- Desktop Firefox
- Desktop Safari
- iPhone SE
- iPhone 15
- Pixel 7
- iPad Mini
- Galaxy Tab S9

These devices were chosen to ensure compatibility across different device sizes/manufacturers. Testing these viewports requires a new test database to be set up which is documented in our README.md. The test cases broadly cover the following areas:

- Ensuring that the landing, login form, register/sign-up form, create lab form, lab page, home page, and reservation schedule all fit the screen properly.
- Ensuring that the buttons on this screen can be interacted with and still perform their intended action.
- The user can navigate/scroll through the pages if needed.

The output after running these test cases can be seen below:



Any testing on real phones/iPads will need to be done in Milestone 3 after we have actually deployed the website.

Threat Model

BenchTime is a React client and an Express API backed by PostgreSQL. The assets we protect are user accounts and passwords, session tokens, lab data (labs, members and equipment), and reservations. Only a lab's manager may edit members, equipment, and reservations for that lab, and only the manager and the lab's members may see it.

Information from the client sent to the server is never trusted. All authentication and authorization is done server side. The server connects to the database using credentials stored in the .env.

Main Attack Surfaces:

Surface	Endpoints	Threats	Mitigations
Auth	POST /api/auth/register, /login, /refresh, /logout	Password guessing, account enumeration, session theft	Login and register are limited to 10 requests per 15 minutes per IP. Passwords are hashed with bcrypt. Login gives the same error, with similar timing, whether or not the username exists. Tokens are kept in httpOnly, SameSite=Lax cookies.
Lab Management	/api/labs/* (create, delete, members, equipment, stats)	Users reaching or changing labs they don't own, members performing manager actions	Every route checks sign-in and role on the server. Every lab query checks ownership or membership, and labs you can't access return 404.
Reservations	/api/labs/:id/.../reservations	Cancelling or editing other users' reservations, double booking	Members can only cancel their own reservations. The database rejects overlapping bookings, even when two arrive at the same time.
API	/api/*	Request flooding, malformed input, leaked error details	300 requests per 15 minutes per IP. All queries are parameterized. Unexpected errors return a generic message, with details logged only on the server.

One potential threat we will have to cover in Milestone 3 is logout doesn’t revoke tokens, so a stolen token is valid for 7 days.

Security Control and Fixes

Security Control

One of the earliest forms of security control that we added to BenchTime was protection against SQL injections. We do this in two ways. The first is by maintaining a repository pattern under the server/src/repositories directory. This is the only location where the server can send SQL queries to the database. By maintaining this pattern, we keep all SQL queries in one centralized location which can easily be audited/tested. The second way is by binding parameters to each query instead of using raw data directly. An example of this can be seen in the reservationRepository.js file:

```
async create({ labId, equipmentId, userId, startTime, endTime }) {
  const result = await pool.query(
    `WITH inserted AS (
      INSERT INTO reservations (equipment_id, user_id, start_time, end_time)
      SELECT e.id, $3, $4, $5
        FROM equipment e
       WHERE e.id = $1 AND e.lab_id = $2 AND e.active
      RETURNING *
    )
    SELECT ${COLUMNS} FROM inserted r JOIN users u ON u.id = r.user_id`,
    [equipmentId, labId, userId, startTime, endTime],
  );
  return result.rows[0] ?? null;
```

This code ensures that even if the parameters are something malicious like “;DROP TABLE users;” the SQL query will treat it like data(causing the query to return an error) instead of part of the SQL query itself. This protects us from any SQL injections.

One of the other forms of security control that we added for this milestone is rate limiting. The issue here is that once we deploy our server in milestone 3, anyone can send endless requests to any of our endpoints effectively DDoSing our server. To help mitigate this, we added rate limiting inside of app.js: which can be seen in the screenshot below:

```
//Setting up endpoint rate limitng.

const FIFTEEN_MINUTES = 15 * 60 * 1000;

const defaultRateLimits = {
  api: { windowMs: FIFTEEN_MINUTES, limit: 300 },
  auth: { windowMs: FIFTEEN_MINUTES, limit: 10 },
};

function createLimiter({ windowMs, limit }) {
  return rateLimit({
    windowMs,
    limit,
    standardHeaders: 'draft-8',
    legacyHeaders: false,
    handler: (req, res) => {
      res.status(429).json({ error: { code: 'RATE_LIMITED', message: 'Too many requests, please try again later.' } });
    },
  });
}

//Builds the Express app. Keeping it separate from server.js for testing.
export function createApp({ rateLimits = defaultRateLimits } = {}) {
  const app = express();

  app.use(express.json());
  app.use(cookieParser());

  app.use('/api', createLimiter(rateLimits.api));
  app.use('/api/auth', createLimiter(rateLimits.auth));
```

It should be noted that the timeout parameters here will most likely need to be tuned more after we deploy to a real server.

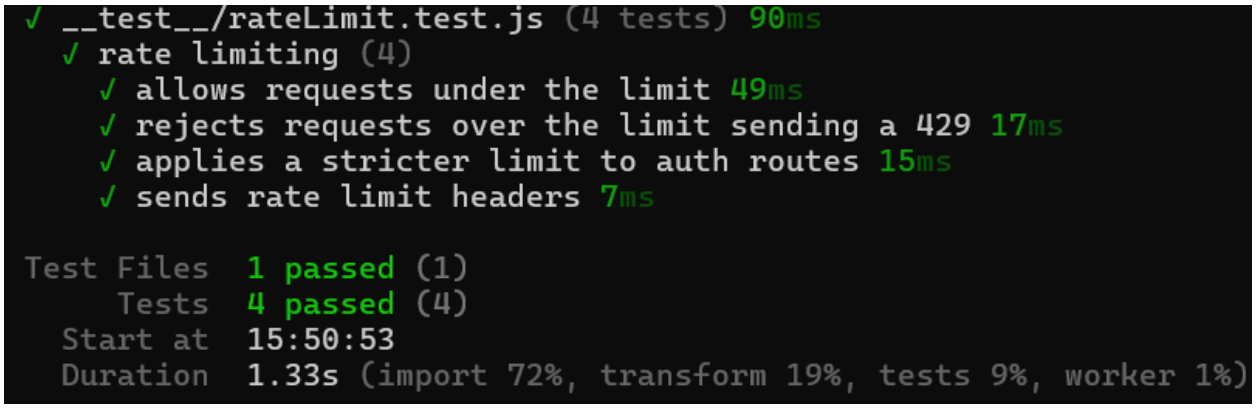
Before and After Evidence

To ensure that our rate limiting is functioning properly, we wrote some test cases. Broadly, these test cases test the following features:

- Ensuring that API requests can still be made if they are sent under the rate limit.
- Ensuring that API requests that are sent too quickly return an error code 429(too many requests).
- Changing/using a stricter rate limit still allows requests to be made and enforces the new limit.

- A request for rate limit headers can be made and the proper information is returned.

The instructions for running these tests can be seen in our README.md, but they all pass as can be seen in the screenshot below:



OWASP Top Ten

We reviewed BenchTime against the OWASP Top 10:2025. The categories most relevant to BenchTime are A01, A05 and A07.

A01 Broken Access Control. This is BenchTime's biggest risk, since users must not reach labs they don't own or belong to. Every protected route checks sign-in and role on the server, and every lab query checks ownership or membership. Labs a user can't access return 404, and our integration tests check access between different users.

A02 Security Misconfiguration. Secrets are kept in a gitignored .env file, with separate secrets for access and refresh tokens, and cookies are marked Secure in production. However, the API sends no security headers, and the X-Powered-By header reveals Express. trust proxy will also need to be configured before we deploy behind a proxy.

A03 Software Supply Chain Failures. Lockfiles pin exact dependency versions, and installs use npm ci. npm audit currently reports 7 known vulnerabilities in the server's dependencies, and we don't check for these automatically yet.

A04 Cryptographic Failures. Passwords are hashed with bcrypt, and tokens are signed with HS256, with the algorithm pinned when verifying. The app isn't served over HTTPS yet; that's planned for deployment in Milestone 3.

A05 Injection. All database queries are parameterized, so user input is never treated as SQL. React escapes everything it displays, which prevents injected scripts.

A06 Insecure Design. Reservation overlap and uniqueness rules are enforced by the database itself, so two simultaneous requests can't get around them. Rate limiting was also built into the design.

A07 Authentication Failures. Login and register are limited to 10 requests per 15 minutes per IP, and login gives the same error whether or not a username exists. Passwords must be 8 to 72 characters, and tokens are stored in httpOnly, SameSite=Lax cookies, with access tokens lasting 15 minutes. Logging out doesn't revoke refresh tokens, which remain valid for up to 7 days, and registration reveals whether a username is taken.

A08 Software or Data Integrity Failures. Tokens are signed, so they can't be altered, and dependencies are installed from lockfiles. We have no CI pipeline yet, which is planned for Milestone 3.

A09 Security Logging and Alerting Failures. Unexpected server errors are logged, but security events such as failed logins and rate limit hits aren't recorded, and there's no alerting.

A10 Mishandling of Exceptional Conditions. A single error handler returns generic messages without exposing internal details, and database constraint errors become clear 400 or 409 responses. Malformed requests, however, are parsed before the rate limiter counts them.

Branch/Commit Conventions

Branches:

‘main’ always holds working code and is never committed to directly. Each new feature starts from a GitHub Issue and gets its own branch from main, named with the issue number and a short description in lowercase with hyphens, for example 25-rate-limiting or 17-admin-edit-res. This links every branch to the work that motivated it.

Pull requests:

When the work is done, the author opens a pull request into main. The other team member reviews the code, runs the feature locally, and checks it against our Definition of Done before merging. Before merging, the author merges the latest main into the branch and reruns the new tests, so changes that pass separately are also tested together. We merge with merge commits, so each pull request appears in the history as Merge pull request #n.

Commits:

Commits are small and focused on one change. Messages use a short summary of what changed.

Changelog and Tagged Releases

A changelog which is updated for every commit/pull request can be found here. This changelog tracks core changes made to the software and corresponds to the items listed in our BACKLOG. Tagged releases are stored on github and so far we have three releases(one for each milestone). They can be found here.

Contribution Statements

Jesse Seidel — Manager utilization statistics. Unit test cases for code created by Luke. Viewport testing/test cases. Component testing/test cases for components created by Luke. In the milestone 2 document: Changes made since Milestone 1, SQA plan, Cross-viewport/cross-platform testing evidence, implemented security control + before and after evidence, and the CHANGELOG.

Luke McArthur— Edit reservation and cancel reservation for managers. Unit test cases for code created by Jesse. Component testing/test cases for components created by Jesse. Integration tests and browser flow tests. In the milestone 2 document: Peer Code Review, Integration Tests, Threat Model, OWASP Top 10, Branch/Commit Conventions.